Part4-HTTP

TODO

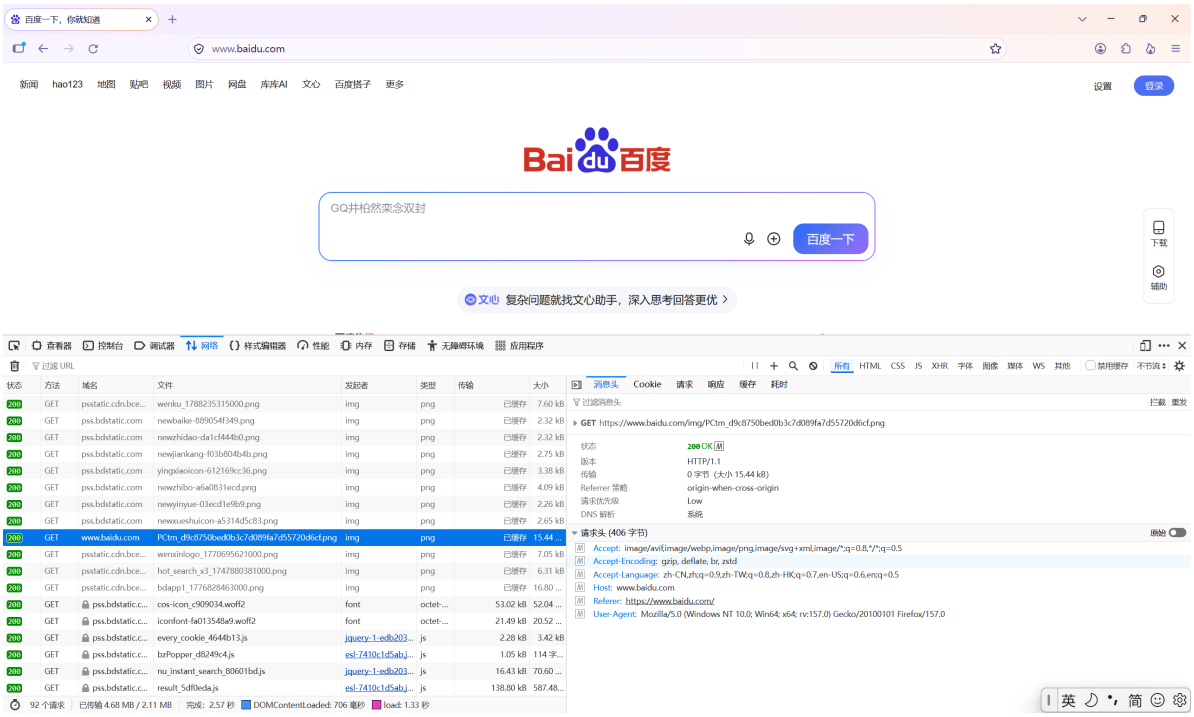
1.HTTP（HyperText Transfer Protocol，超文本传输协议）是互联网上应用最广泛的协议之一，用于在客户端（如浏览器）和服务器之间传输超文本（如网页）。

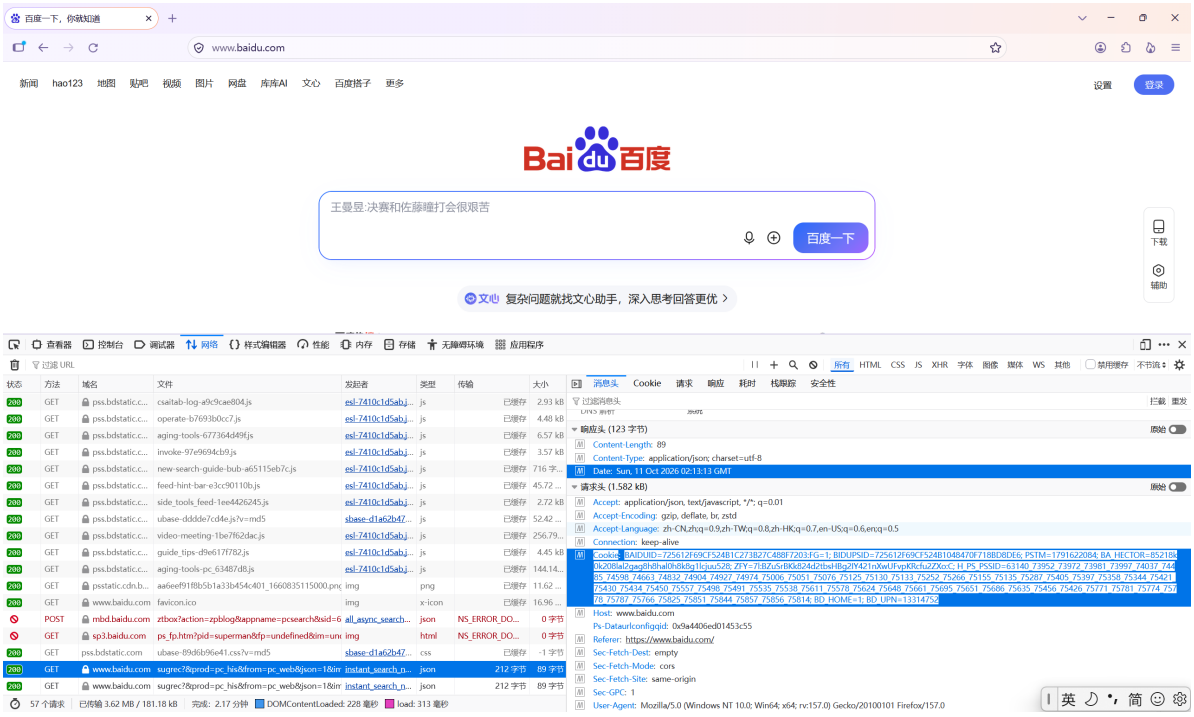
2.

开头	含义	理解
2xx	成功	一切正常
3xx	重定向	去别的地方
4xx	客户端错	你（请求方）的问题
5xx	服务端错	我（服务器）的问题

日常最常打交道的就那几个：**200、301、302、304、400、401、403、404、500、502、504。**

Task1





由上图可知：

Method是GET

Header中有：

Accept	**application/json, text/javascript, /; q=0.01**
Accept-Encoding	gzip, deflate, br, zstd
Accept-Language	zh-CN,zh;q=0.9,zh-TW;q=0.8,zh-HK;q=0.7,en-US;q=0.6,en;q=0.5
Connection	keep-alive

Host	**www.baidu.com**
Ps-Dataurlconfigqid	0x9a4406ed01453c55
Referer	https://www.baidu.com/
Sec-Fetch-Dest	empty
Sec-Fetch-Mode	cors
Sec-Fetch-Site	same-origin
Sec-GPC	1
User-Agent	Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:157.0) Gecko/20100101 Firefox/157.0

其中：

Accept表示可接受的相应方式

Accept-Encoding表示可接受的压缩方式

Accenpt-Language表示可接受的语言

Host表示目标主机

Referer表示来源页面

User-Agent表示客户端标识

connection表示连接复用

cookie表示携带的会话信息

Sec系列表示浏览器安全策略标识

服务器返回内容：

Content-Length	**89**
Content-Type	application/json; charset=utf-8
Date	Sun, 11 Oct 2026 02:13:13 GMT

content系列分别标识返回内容的长度、类型

date标识时间

一次HTTP请求包含的部分及作用如下：

请求部分：

部分　　作用
请求行　方法 + 路径 + 协议版本
请求头　描述请求的元信息
空行　　分隔
请求体　提交的数据（POST 才有）

相应部分：

部分　　作用
状态行　协议版本 + 状态码
响应头　描述响应的元信息
空行　　分隔
响应体　实际返回的数据（这里 89 字节 JSON）

常见 HTTP 方法及场景:

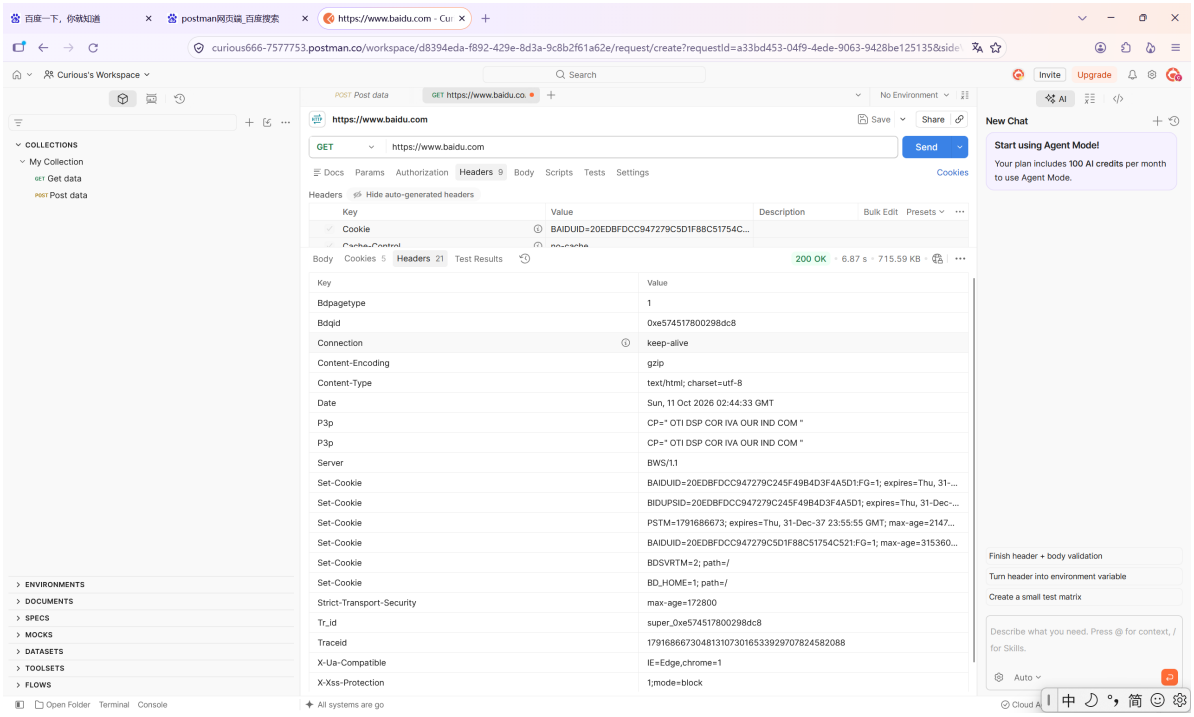
方法	**语义**	**典型场景**
GET	获取资源	打开网页、加载脚本/图片、查询数据（截图里绝大多数）
POST	提交/创建	登录、下单、提交表单（截图里 `mbd.baidu.com` 那条）
PUT	全量更新	替换整条记录
PATCH	局部更新	只改某几个字段

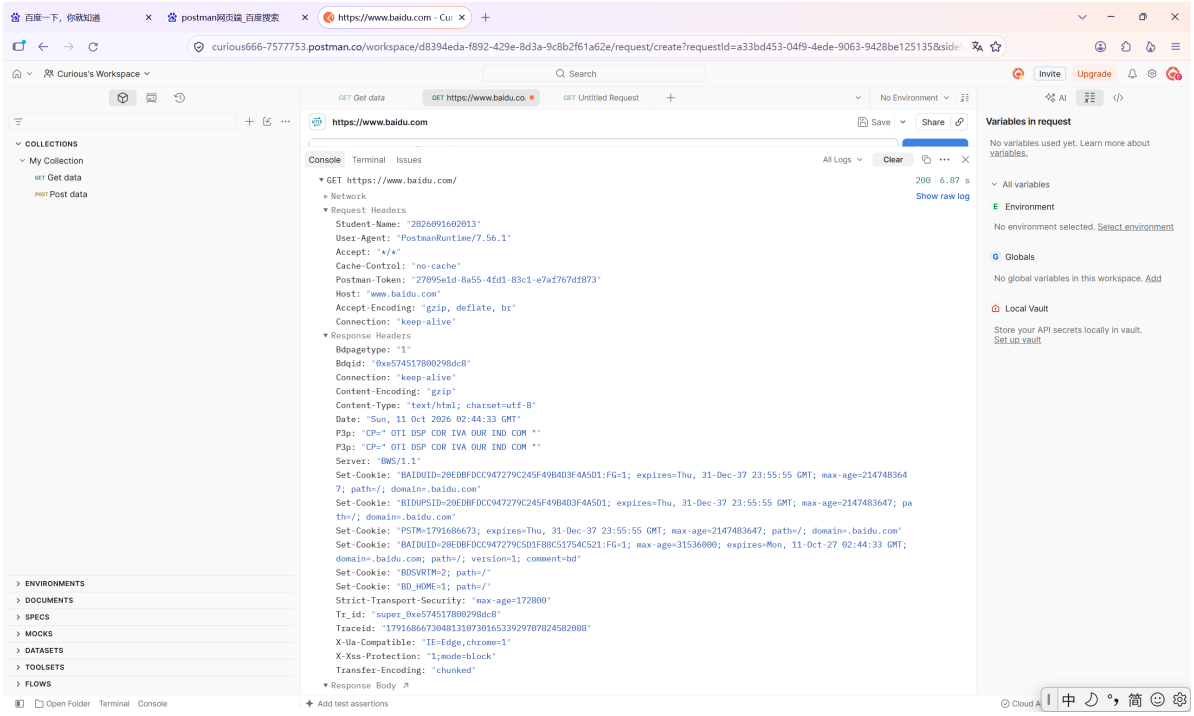
方法	语义	典型场景
DELETE	删除	删除用户、删除文章
HEAD	只要响应头	检查资源是否存在
OPTIONS	查询支持的方法	CORS 跨域预检

Task2：修改请求 Header

发送请求：

返回结果：





关于思考题，我的看法如下：

1.Header本质是元数据，它用来描述这次请求的附加信息，而不是要传输的实际数据。

比如，Get告诉对方我要什么，Header就表示我是谁，我能接受什么，我在哪，以什么样的身份……从而对请求进行具体说明。

总而言之，Header 让服务器能做出正确响应，而不是盲目地返回一个默认格式。

2.Host是一个强制请求头，没有便直接400（返回默认值）；就其他Header来说，它们其实是可选的，只有在特定场景才会用。所以没有Header的话，服务器在协议层面就不能正确处理请求；其次，要解析JSON文件、传输文件、鉴权、跨域等的请求，在没有Header时在业务层面上不能被正确处理。（也许本题提问的方式不对或者我理解的不够，在作答时总感觉怪怪的，不过我还是按我的理解给出了相对合理的答案:）希望不会被扣分）

3.其他常见Header（Task1中的都算吧）：

Accept系列，用来规范服务器响应的内容

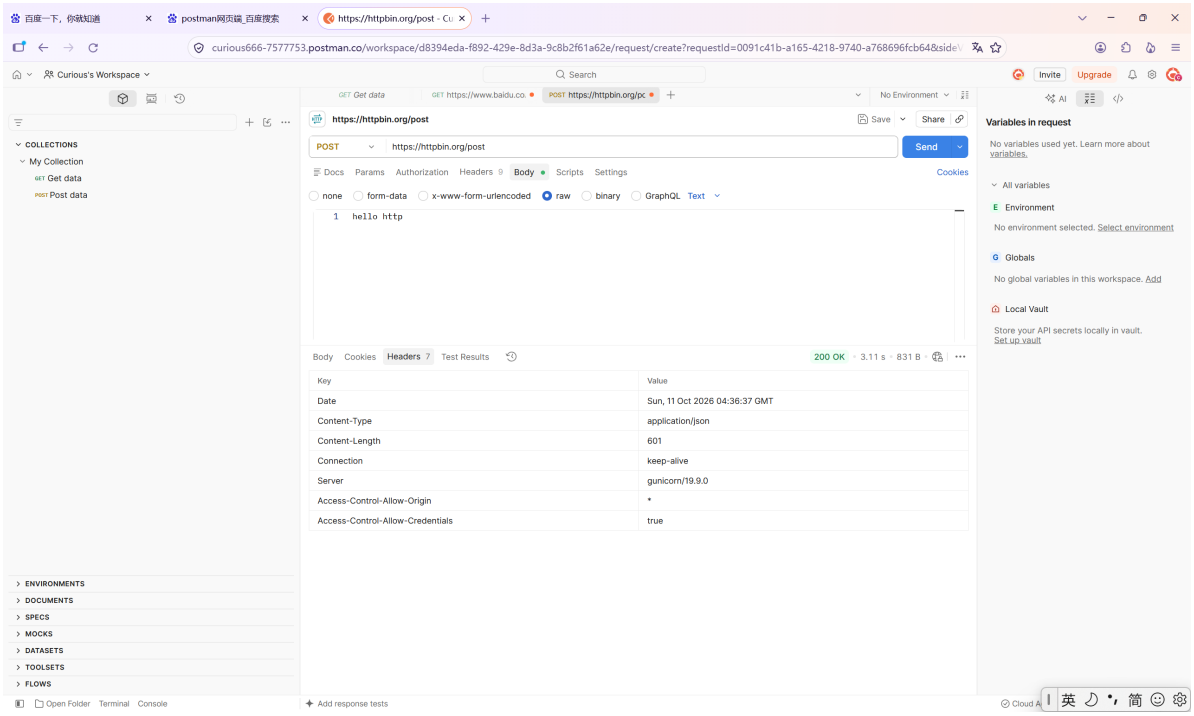
Cookei、User-Agent等用于身份认证

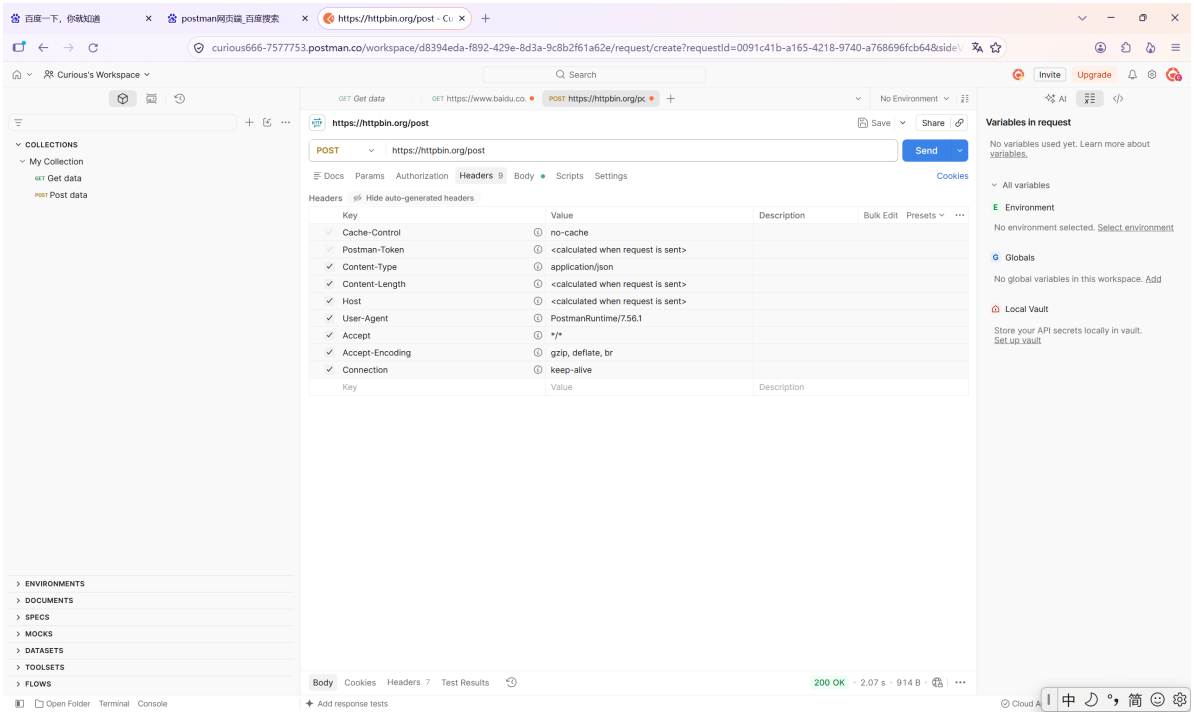
以及Content系列用于描述请求体格式

Task3：发送 POST 请求提交数据

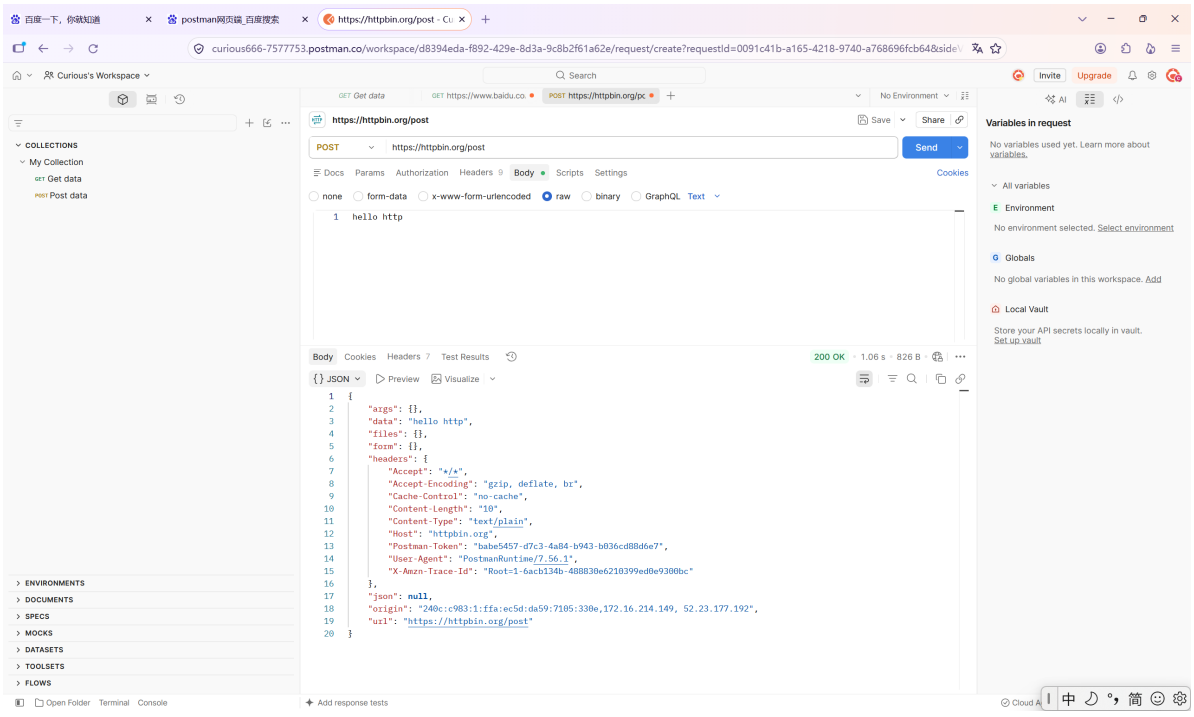
Task3-1：发送普通文本

请求：



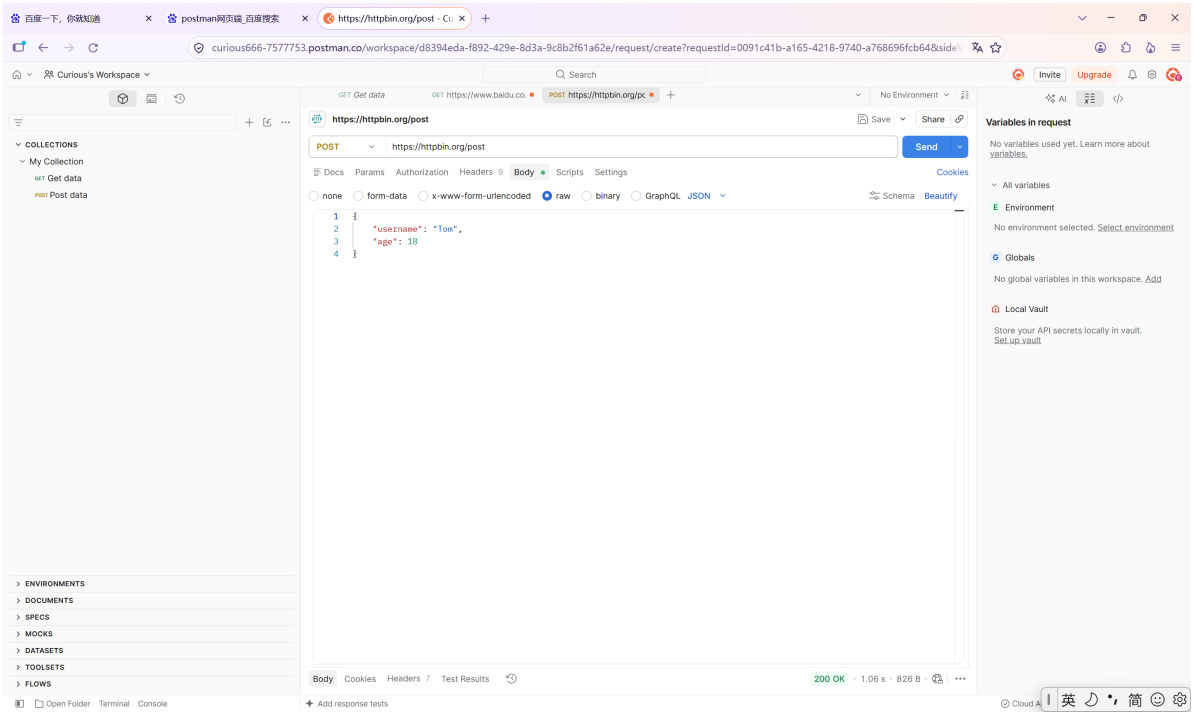


响应：

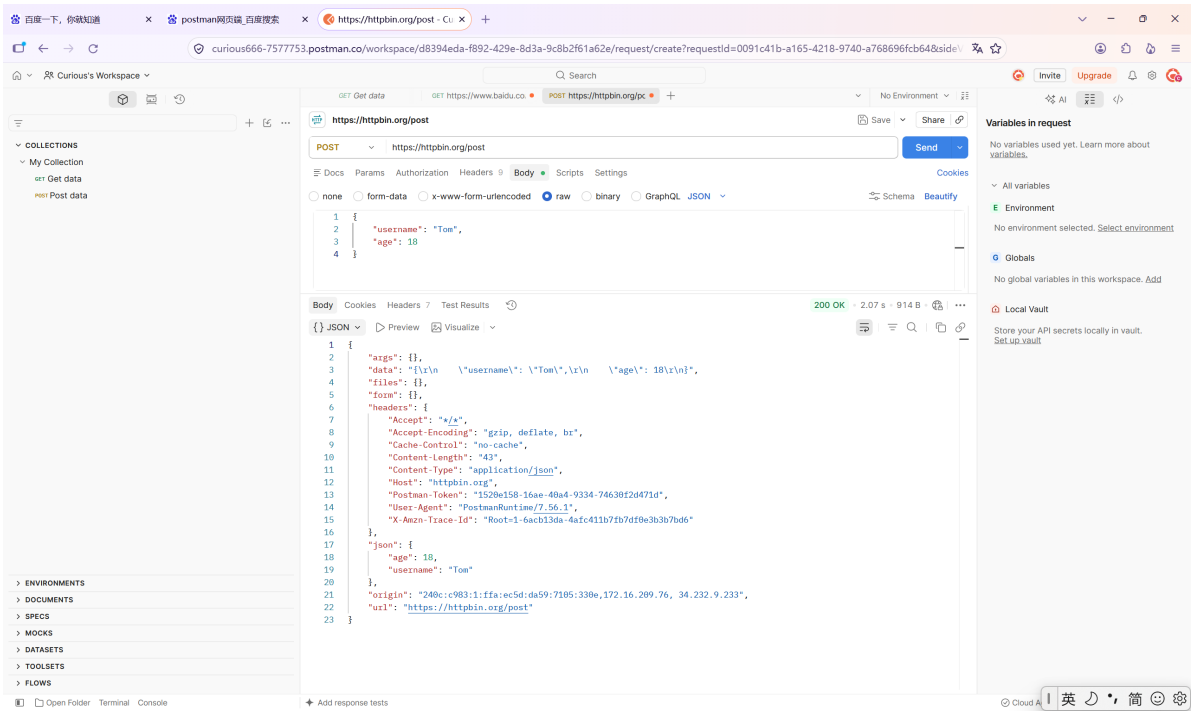


Task3-2：发送 JSON 数据

请求：



响应：



思考题：

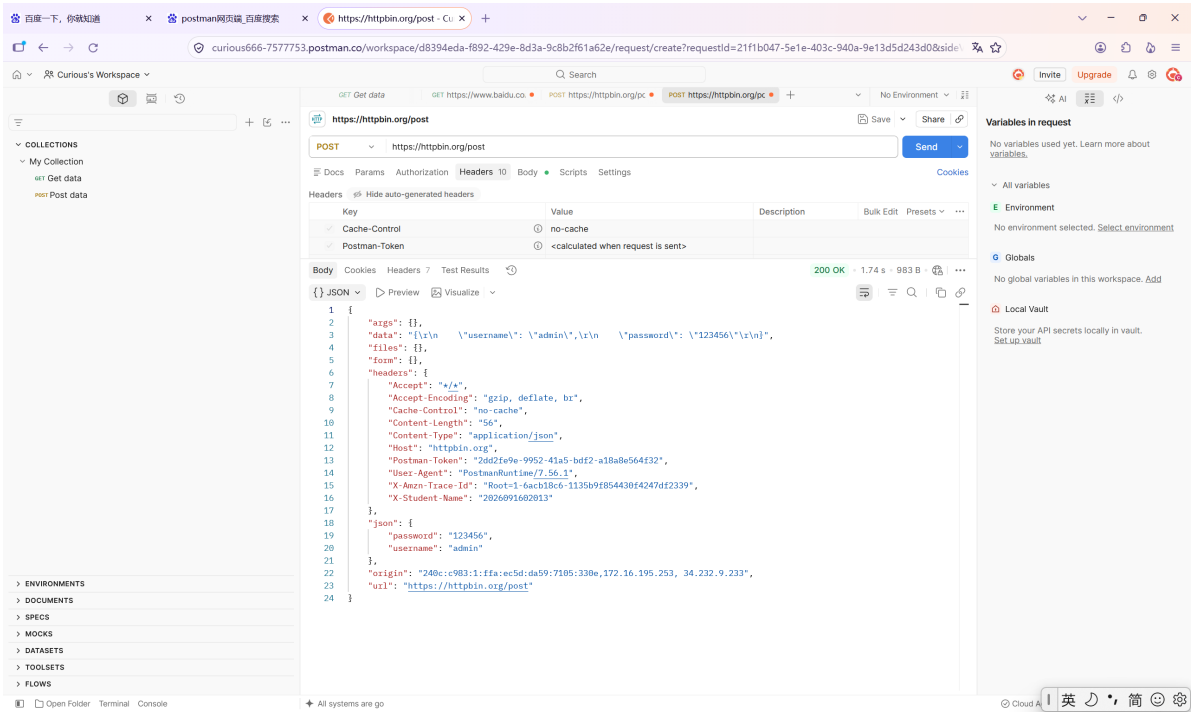
1.GET请求只用于获取资源，不适合向服务器提交数据，因为它传数据不仅长度受限，而且极易暴露。而POST请求可以向服务器上传数据，它没有GET的各种缺点，更加安全方便。

2.设置这个请求头是为了描述上传的数据类型，从而让服务端能正确的解析数据。否则服务端只能收到原始数据，没法调用工具进行解析。

Task4：模拟一个简单登录请求

请求构造：

响应：



思考题：

1.显然，登录请求需要上传用户名、密码等数据，当然应用POST，不适合用GET

2.设置content-type请求头能让服务器正确地解析JSON文件，从而拿到登录数据

3.URL是暴露在外的，不能携带用户名和密码